



07-08 API / Express

Prepared by Surajai Chamroensat



Agenda

- What is an API?
- What is web service?
- What is Express.js?
- Setting up an Express App
- Http Method
- Http Status
- Routing in Express
- Validation
- Environment Variable
- CORS
- API Document (Swagger)
- Testing APIs

What is an API?

- API = Application Programming Interface
- A set of rules that allows software components to communicate
- Acts as a bridge between client and server
- Types:
 - Web APIs (REST, SOAP, GraphQL)
 - Library APIs (functions in a programming library)
- Benefits:
 - Reusability
 - Abstraction of complexity
 - Faster development
-

What is web service?

- A software system designed to support interoperable machine-to-machine interaction over a network
- Enables applications to communicate and share data regardless of platform or language
- Uses standard protocols (HTTP, HTTPS)
- Platform-independent
- Often exposes functionality via APIs

Is API a Web Service

- API (Application Programming Interface) is the broad concept — it's any defined way for one piece of software to interact with another.
 - Can be local (e.g., a database driver, OS function calls) or remote (over a network).
 - Doesn't have to use the web at all.
- Web Service is a specific kind of API that is always network-based and accessible over standard web protocols like HTTP/HTTPS, SOAP, or REST.
 - Designed for machine-to-machine communication over the internet or an intranet.
 - Examples: SOAP services, RESTful APIs exposed over HTTP.

SOAP (Simple Object Access Protocol)

- A protocol for exchanging structured information in web services.
- Always uses XML for requests and responses.
- Strict rules and standards (formal contract via WSDL – Web Services Description Language)
- Works over multiple protocols (HTTP, SMTP, TCP, etc.)
- Built-in error handling and security features (WS-Security)
- Supports stateful and stateless operations

REST (Representational State Transfer)

- An architectural style for designing networked applications.
- Commonly uses JSON, but can also use XML, HTML, or plain text.
- Stateless — each request contains all needed information
- Uses standard HTTP methods: GET, POST, PUT, DELETE, PATCH
- Resource-based — each resource has a unique URI
- Lightweight, easy to learn, and widely adopted
- Typical use for
 - Web and mobile applications
 - Public APIs (e.g., Twitter API, GitHub API)
 - Microservices architectures

REST v.s. SOAP

Feature	REST (Representational State Transfer)	SOAP (Simple Object Access Protocol)
Protocol	Architectural style (uses HTTP)	Strict protocol
Data Format	JSON, XML, others	XML only
Ease of Use	Lightweight, easy to learn	More complex, verbose
Performance	Faster, less bandwidth	Slower due to XML overhead
Flexibility	Flexible with formats & standards	Strict standards & rules
Use Cases	Web/mobile apps, microservices	Enterprise systems, financial services

API Data Format

- JSON (JavaScript Object Notation)
 - Lightweight, human-readable, easy to parse
 - Most popular for REST APIs
- XML (Extensible Markup Language)
 - Structured, supports complex data
 - Common in SOAP APIs
- Others:
 - YAML (configuration)
 - Protocol Buffers (binary, high performance)

What is Express.js?

- A minimal and flexible Node.js web application framework
- Simplifies building web servers and APIs
- Key Features:
 - Fast routing system
 - Middleware support
 - Easy integration with databases
 - Large ecosystem of plugins

Setting up an Express App

- Installation
 - `npm install express`
- Simple Web API
 - `const express = require('express');`
 - `const app = express();`
 - `app.get('/', (req, res) => res.send('Hello World'));`
 - `app.listen(3000, () => console.log('Server running on port 3000'));`

HTTP Method

- GET
- POST
- PUT
- DELETE
- PATCH
- DELETE
- OPTIONS

Postman

- <https://www.postman.com/downloads/>

HTTP Status

- 1xx – Informational
 - 100 Continue – Request headers received, client should proceed with request body
 - 101 Switching Protocols – Server is switching to a different protocol as requested
- 2xx – Success
 - 200 OK – Request succeeded (most common success code)
 - 204 No Content – Request succeeded, no content to return
- 3xx – Redirection
 - 301 Moved Permanently – Resource has a new permanent URL
 - 302 Found – Temporary redirect to another URL

HTTP Status

- 4xx – Client Errors
 - 400 Bad Request – Server cannot process request due to client error (e.g., malformed syntax)
 - 401 Unauthorized – Authentication required or failed
 - 403 Forbidden – Server understood request but refuses to authorize it
 - 404 Not Found – Requested resource not found
 - 405 Method Not Allowed – HTTP method not supported for this resource
- 5xx – Server Errors
 - 500 Internal Server Error – Generic server-side error
 - 502 Bad Gateway – Invalid response from upstream server
 - 503 Service Unavailable – Server temporarily overloaded or down for maintenance
 - 504 Gateway Timeout – Upstream server failed to respond in time

Routing in Express

- `app.get('/users', getUsers);`
- `app.post('/users', createUser);`
- `app.put('/users/:id', updateUser);`
- `app.delete('/users/:id', deleteUser);`

Error handling

```
app.use((err, req, res, next) => {  
  res.status(500).json({ error: err.message });  
});
```

Validation

- Use Libraries: express-validator, Joi
- npm install joi
- npm install express-validator

```
const { body } = require('express-validator');  
app.post('/user', body('email').isEmail(), handler);
```

<https://express-validator.github.io/docs/api/validation-chain#standard-validators>

CORS (Cross-origin resource sharing)

- A browser security feature that controls which domains can access your API.
- Prevents unauthorized cross-site requests
- Needed when frontend & backend are on different domains/ports
- Installation
 - `npm install cors`
- Implementation
 - `const cors = require('cors');`
 - `app.use(cors({ origin: 'http://yourfrontend.com' }));`

API Document

- Swagger Installation
 - npm install swagger-ui-express swagger-jsdoc

```
const express = require('express');
const swaggerUi = require('swagger-ui-express');
const swaggerJsdoc = require('swagger-jsdoc');
```

```
const app = express();
const port = process.env.PORT || 3000;
```

```
// Swagger configuration
const swaggerOptions = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'My API',
      version: '1.0.0',
      description: 'API documentation with Swagger',
    },
    servers: [
      { url: 'http://localhost:3000' }
    ],
  },
  apis: ['./routes/*.js'], // Path to your route files with JSDoc comments
};

const swaggerSpec = swaggerJsdoc(swaggerOptions);

// Serve Swagger UI
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

References

- <https://expressjs.com/>